

V2X Attack Detection — Full Pipeline Walkthrough

Machine Learning Pipeline: Data Cleaning → EDA → Feature Engineering → Model Training

1. Project Overview

This project builds a two-stage machine learning system to detect cyberattacks in V2X (Vehicle-to-Everything) networks. In V2X, vehicles continuously broadcast messages containing their position, speed, and heading to nearby vehicles and infrastructure. Attackers can inject false or manipulated messages to cause accidents, traffic disruption, or denial of service.

The system reads incoming V2X messages and first decides: is this message an attack? If yes, it then classifies what type of attack it is — which determines the appropriate incident response in a SIEM (Security Information and Event Management) system.

2. The Dataset

The VeReMi (Vehicular Reference Misbehavior) dataset is the standard academic benchmark for V2X security research. It was generated using a vehicular network simulator where both normal and malicious vehicles were modelled.

Property	Value
Total rows	2,269,540
Number of columns	19
Number of classes	20 (1 Benign + 19 attack types)
Rows per class	113,477 (pre-balanced)

2.1 Feature Descriptions

Each row represents one V2X broadcast message received by a vehicle. Positions and velocities are given in a 2D Cartesian plane where:

- **X-axis (suffix _0)** — East–West direction. Positive values mean the vehicle is East of the origin.
- **Y-axis (suffix _1)** — North–South direction. Positive values mean the vehicle is North of the origin.

So pos_0 and pos_1 together give the 2D grid coordinate of the vehicle. spd_0 and spd_1 are the X and Y components of the velocity vector — you combine them as $\sqrt{\text{spd}_0^2 + \text{spd}_1^2}$ to get the actual scalar speed.

Column	Full Name	Meaning
rcvTime	Receive Time	Timestamp (seconds) when the message arrived at the receiver
pos_0	Position X	Claimed X coordinate (East–West, in metres)
pos_1	Position Y	Claimed Y coordinate (North–South, in metres)
pos_noise_0	Position Noise X	Sensor measurement noise on the X position
pos_noise_1	Position Noise Y	Sensor measurement noise on the Y position
spd_0	Speed X-component	Claimed velocity in the East–West direction (m/s)
spd_1	Speed Y-component	Claimed velocity in the North–South direction (m/s)
spd_noise_0	Speed Noise X	Sensor noise on the X speed component
spd_noise_1	Speed Noise Y	Sensor noise on the Y speed component
acl_0	Acceleration X	Claimed acceleration in the East–West direction (m/s^2)
acl_1	Acceleration Y	Claimed acceleration in the North–South direction (m/s^2)
acl_noise_0	Acceleration Noise X	Sensor noise on X acceleration
acl_noise_1	Acceleration Noise Y	Sensor noise on Y acceleration
hed_0	Heading X-component	X component of the unit direction vector the vehicle faces
hed_1	Heading Y-component	Y component of the unit direction vector the vehicle faces
hed_noise_0	Heading Noise X	Sensor noise on heading X
hed_noise_1	Heading Noise Y	Sensor noise on heading Y
ReceiverID	Receiver Vehicle ID	String ID of the vehicle that received the message
AttackerType	Label	Ground truth — Benign or one of 19 attack types

3. Data Cleaning & Preprocessing

3.1 Encode ReceiverID

ReceiverID is a string identifier. Scikit-learn and gradient boosting models require numeric inputs. LabelEncoder converts each unique vehicle ID to an integer. The fitted encoder is saved as

receiver_encoder.pkl so the live simulation applies the identical mapping and the model sees the same integer representation.

3.2 Create Label Columns

Two separate target columns are created because two separate models will be trained:

- **Y_binary** — 0 = Benign, 1 = Any attack. Used by the binary classifier.
- **Y_multi** — Integer-encoded attack type (0–19). Used by the multiclass classifier.

3.3 Why No Missing Value Treatment?

The VeReMi dataset is simulation-generated, so every row is complete. No imputation was needed. The only NaN values that appear are introduced intentionally by the .diff() operation in feature engineering (first row per vehicle has no previous message to compare against) — those rows are dropped with dropna().

4. Exploratory Data Analysis (EDA)

4.1 Class Distribution

The dataset was found to be perfectly balanced: exactly 113,477 rows per class across all 20 classes. This is unusual in real-world security datasets (attacks are rare) but expected here because the dataset was synthetically generated with this balance in mind. This means no oversampling/undersampling was needed at the raw class level.

4.2 Attack vs Benign Imbalance

Despite per-class balance, there is a severe overall imbalance in the attack/benign split:

Category	Count	Percentage
Benign	113,477	5%
Attack	2,156,063	95%
Total	2,269,540	100%

This was handled during dataset construction: for the binary model, a 1:1 sample was drawn to give the model equal exposure to both classes.

4.3 Key Observations per Attack Type

Attack Type	Characteristic Signal
DoS / DoSRandom / DoSDisruptive	Very high message rate — vehicles flood the channel
ConstPos / ConstPosOffset	Position stays constant or offset by a fixed value — no movement
RandomPos / RandomPosOffset	Position jumps erratically — impossible physics
ConstSpeed / RandomSpeed	Speed component frozen or randomly set

DataReplay	Messages are copies of old valid messages — rcvTime anomaly
DelayedMessages	Messages arrive with large time lag relative to content timestamp
EventualStop	Speed gradually decreases to zero — mimics a breakdown
Sybil variants	Same attacker pretends to be multiple vehicles simultaneously

5. Feature Engineering

Raw per-message features are enough to separate benign from attack (binary task) but not enough to distinguish attack types from each other. The key insight is that attack behaviour is visible in how messages change over time, not just in a single message. Feature engineering was applied only to attack data (used by the multiclass model).

Data is first sorted by ReceiverID and rcvTime to establish temporal order per vehicle.

5.1 Delta Features

Computed using pandas `.groupby("ReceiverID").diff()` — the change between consecutive messages from the same vehicle:

Feature	Formula	What it detects
delta_time	$rcvTime[i] - rcvTime[i-1]$	Abnormally small = DoS flooding (too many messages)
delta_pos_0	$pos_0[i] - pos_0[i-1]$	How far the vehicle moved East–West
delta_pos_1	$pos_1[i] - pos_1[i-1]$	How far the vehicle moved North–South
delta_spd_0	$spd_0[i] - spd_0[i-1]$	Speed change — sudden jumps = SpeedManip
delta_hed_0	$hed_0[i] - hed_0[i-1]$	Heading change — impossible turns
delta_dist	$\sqrt{(\delta pos_0)^2 + (\delta pos_1)^2}$	Actual Euclidean distance moved
expected_dist	$ spd_0 \times \delta time $	Distance the vehicle should have moved
dist_error	$\delta dist - \text{expected_dist}$	Large = position doesn't match claimed speed (PositionAttack)
delta_pos_noise_0	$pos_noise_0[i] - pos_noise_0[i-1]$	Noise pattern change
delta_pos_noise_1	$pos_noise_1[i] - pos_noise_1[i-1]$	Noise pattern change

5.2 Sliding Window Features

Rolling statistics over the last 10 messages per vehicle (window=10). These capture patterns invisible in any single message:

Feature	Computation	What it detects
pos_std	Rolling std of pos_0 over 10 msgs	High = vehicle position jumping randomly (RandomPos)
spd_std	Rolling std of spd_0 over 10 msgs	High = speed fluctuating randomly (RandomSpeed)
hed_std	Rolling std of hed_0 over 10 msgs	High = impossible heading changes
msg_rate	1 / mean(delta_time) over 10 msgs	Very high = DoS flooding; low = DelayedMessages

After delta computation, rows with NaN (the first message per vehicle) are dropped, leaving 2,156,061 rows across 19 attack types.

6. Class Grouping (19 → 6)

Many of the 19 attack types are behaviourally identical from the perspective of a single receiver — a model cannot reliably tell them apart because they produce the same feature signatures.

Grouping them improves model reliability without losing operational value (the response action is the same for the whole group).

Group	Original Attack Types	Reason for Grouping
DoSFamily	DoS, DoSRandom, DoSDisruptive, Disruptive, DoSRandomSybil, DoSDisruptiveSybil, DataReplaySybil, GridSybil	All cause channel flooding or message overload — identical msg_rate signature
PositionAttack	ConstPos, ConstPosOffset, RandomPos, RandomPosOffset	All report wrong position — same dist_error / pos_std pattern
SpeedManip	ConstSpeed, ConstSpeedOffset, RandomSpeed, RandomSpeedOffset	All report wrong speed — same delta_spd / spd_std pattern
ReplayAttack	DataReplay	Replays old valid messages — detected via rcvTime vs content timestamp anomaly
DelayedMessages	DelayedMessages	Artificially delayed — low msg_rate, large delta_time

EventualStop	EventualStop	Speed gradually decreasing — unique temporal pattern in spd_std
--------------	--------------	---

Group	Row Count
DoSFamily	907,816
PositionAttack	453,908
SpeedManip	453,906
EventualStop	113,477
DelayedMessages	113,477
ReplayAttack	113,477

7. Building the Training Datasets

7.1 Binary Dataset

Uses the original raw 17 features (no engineering needed — binary task is simple enough).

1. Take all 113,477 Benign rows.
2. Randomly sample 113,477 rows from the 2.1 million attack rows.
3. Shuffle and combine → 226,954 rows, 50/50 split.

7.2 Multiclass Dataset

Uses all 31 features (17 raw + 14 engineered).

4. Find the smallest group size: 113,477 (EventualStop, DelayedMessages, ReplayAttack).
5. Sample exactly 113,477 rows from each of the 6 groups.
6. Shuffle and combine → 680,862 rows, perfectly balanced.

Dataset	Rows	Features	Train	Test
Binary	226,954	17	181,563	45,391
Multiclass	680,862	31	544,689	136,173

8. Train/Test Split and Feature Scaling

Both datasets use an 80/20 train/test split with stratify=Y, which ensures the class proportions are identical in both splits — important for a fair evaluation.

StandardScaler is applied to normalise features to zero mean and unit variance. The scaler is fit only on the training set and then applied to both train and test. Fitting on test data would cause

data leakage — the model would indirectly see test statistics during training. Both scalers are saved to disk for use by the simulation.

Note: gradient boosted trees (XGBoost, LightGBM) are mathematically scale-invariant because they split on thresholds, not distances. Scaling is applied here for consistency and to keep the simulation pipeline uniform between the binary and multiclass models.

9. Model Training

9.1 Binary Classifier — XGBoost

Answers: is this message an attack or not?

Hyperparameter	Value	Purpose
n_estimators	200	Number of trees — enough for a simple binary task
max_depth	6	Tree depth — moderate, prevents overfitting
learning_rate	0.1	Step size — standard starting value
subsample	0.8	Use 80% of rows per tree — reduces variance
colsample_bytree	0.8	Use 80% of features per tree — reduces correlation between trees

Binary Results

Metric	Value
Accuracy	100%
F1 Score	1.00
Precision	1.00
Recall	1.00
False Positive Rate	0.0000
False Negative Rate	0.0000
Training Time	0.7 seconds

The perfect score reflects the clean simulation-generated data — the kinematic features (position, speed, heading values) contain a very sharp signal separating Benign from Attack. This is expected for synthetic datasets with controlled attack injection.

Confusion Matrix

	Predicted: Benign	Predicted: Attack
Actual: Benign	TN = 22,696	FP = 0
Actual: Attack	FN = 0	TP = 22,695

9.2 Multiclass Classifier — XGBoost vs LightGBM

Answers: which type of attack is it?

Both models were trained and the better one was saved.

Hyperparameter	XGBoost	LightGBM	Purpose
n_estimators	5000	5000	Max trees — early stopping decides actual count
max_depth	8	8	Deeper trees for complex multi-class boundaries
learning_rate	0.05	0.05	Slower learning rate to use more trees effectively
num_leaves	—	127	LightGBM-specific: leaf-wise growth for efficiency
subsample	0.8	0.8	Row subsampling
colsample_bytree	0.8	0.8	Feature subsampling
gamma / min_child	0.1 / 5	— / 20	Regularisation to avoid overfitting fine-grained classes
Early stopping	100 rounds	100 rounds	Stop if no improvement for 100 trees

Multiclass Results

Model	Accuracy	F1 (weighted)	Training Time
XGBoost	66.79%	66.76%	~22 minutes
LightGBM	68.85%	68.87%	~31 minutes

LightGBM was saved as model_multiclass.pkl. Its leaf-wise tree growth creates finer decision boundaries than XGBoost's level-wise growth, which benefits this problem where attack groups have overlapping feature distributions.

Per-Class Breakdown (LightGBM)

Class	Precision	Recall	F1	Support
DelayedMessages	0.77	0.75	0.76	22,695
DoSFamily	0.59	0.71	0.64	22,695
EventualStop	0.78	0.74	0.76	22,696
PositionAttack	0.62	0.58	0.60	22,696
ReplayAttack	0.74	0.76	0.75	22,696
SpeedManip	0.65	0.59	0.62	22,695
Overall	0.69	0.69	0.69	136,173

DoSFamily has the lowest precision (0.59) because it merges 8 original classes with diverse feature patterns. PositionAttack has the lowest recall (0.58) because some position manipulation patterns resemble benign GPS drift. On a 6-class problem where random chance is 16.7%, achieving 69% is strong performance.

10. The Two-Stage Inference Pipeline

In the live simulation, every incoming V2X message passes through both models in sequence:

Stage	Model	Output	Action
Stage 1	Binary (XGBoost)	Benign	Message passes through — no alert
Stage 1	Binary (XGBoost)	Attack	Pass to Stage 2 for classification
Stage 2	Multiclass (LightGBM)	DoSFamily	Trigger flooding/Sybil alert
Stage 2	Multiclass (LightGBM)	PositionAttack	Trigger GPS spoofing alert
Stage 2	Multiclass (LightGBM)	SpeedManip	Trigger speed manipulation alert
Stage 2	Multiclass (LightGBM)	ReplayAttack	Trigger replay detection alert
Stage 2	Multiclass (LightGBM)	DelayedMessages	Trigger delay injection alert
Stage 2	Multiclass (LightGBM)	EventualStop	Trigger ghost vehicle alert

11. Saved Artefacts

File	Purpose
model_binary.pkl	Trained XGBoost binary classifier
model_multiclass.pkl	Trained LightGBM multiclass classifier

scaler_binary.pkl	StandardScaler fitted on binary training data
scaler_multi.pkl	StandardScaler fitted on multiclass training data
receiver_encoder.pkl	LabelEncoder for ReceiverID strings → integers
label_encoder.pkl	LabelEncoder for all 20 original AttackerType labels
label_encoder_multi.pkl	LabelEncoder for the 6 grouped AttackGroup labels
label_mapping.json	Human-readable integer → attack name map (20 classes)
label_mapping_multi.json	Human-readable integer → attack group map (6 classes)

12. Key Design Decisions & Why

Decision	Reason
Two separate models (binary + multiclass)	Binary is near-perfect and fast; multiclass is harder. Separating them avoids the harder problem degrading the easier one.
Feature engineering only for multiclass	Binary task is solvable from raw features alone. Engineering adds complexity and data loss (dropna) not worth paying for binary.
Class grouping 19→6	Many attack types are indistinguishable from per-receiver features. Grouping improves reliability and maps to distinct response actions.
LightGBM over XGBoost for multiclass	Leaf-wise growth creates finer decision boundaries. 2% accuracy gain on a 6-class problem is operationally significant.
StandardScaler despite using tree models	Consistency across binary and multiclass pipelines in simulation. Also future-proofs if non-tree models are added.
Stratified train/test split	Ensures evaluation reflects true class distribution — without it, a rare class could be entirely absent from the test set.